Detecting Sentiment Drift and Bot-Like Text Patterns in Social Media Data

Machine Learning Fundamentals Final Project Report

Jarren Seson, Bryan Nakasone

Course: ICS 435

Date:4/29/26

Repository: https://github.com/BryanNak/SentimentAnalysis

1. Introduction

1.1 Problem Statement

This project aims to quantify public opinion shifts on social media and correlate emotional reactions to real-world events over time. E-commerce and social platforms currently suffer from highly inflated or deflated data due to bot networks. Because of the massive scale of unstructured text generated daily, manual data aggregation is not computationally viable. Furthermore, traditional analysis often fails to account for adversarial contamination, resulting in sentiment metrics that are heavily distorted by bot-generated posts rather than reflecting true human sentiment.

1.2 Proposed Solution

Our solution is a machine learning pipeline that can be summarized as follows:

1. Scrape unstructured social media data from platforms like Bluesky based on user-supplied keywords and time ranges
2. Identify and remove bot-generated posts to ensure the integrity of the data
3. Run natural language processing on the clean data to classify human emotion
4. Aggregate the predictions to visualize continuous sentiment drift over time

2. Data

2.1 Data Sources Overview

Data Source	Purpose in Project	Main Fields
Bot-identification dataset	Training/evaluating bot identification model	screen_name, text, account.type, class_type
TweetEval sentiment data	Fine-tuning and evaluating the RoBERTa sequence classification head	text, label
Scraped Bluesky posts (via Bluesky API)	Extraction of unlabelled data for mapping continuous sentiment drift	keyword, post_uri, cid, author_did, author_handle, post_text, created_at_raw, created_at_iso , indexed_at, reply_parent_uri, reply_root_uri, like_count, repost_count, reply_count, quote_count, source_url

2.2 Bot-Identification Dataset

For the bot detection phase, the project utilizes the TweepFake deepfake text detection dataset. This dataset contains tweets from 23 distinct bot accounts designed to imitate 17 human users, alongside authentic tweets from those targeted humans. The dataset consists of 25,572 rows and is perfectly balanced (50% human-generated, 50% bot-generated). The primary target variable is the account type (human or bot), and the raw text of the tweet serves as the primary input feature.

Feature	**Description**
screen_name	Twitter username associated with the tweet
text	Tweet text used as the primary model input
account.type	Target label indicating whether the account is bot or human
class_type	Generation method or class type, such as Markov chains, RNN, RNN + Markov, LSTM, GPT-2, or human

2.3 Sentiment Dataset and Sentiment Labels

For sentiment analysis, we utilize the **cardiffnlp/tweet_eval** dataset to fine-tune our RoBERTa-base classification head. The relevant task classifies short-form social media text into three distinct categories. These numeric labels (0=Negative, 1=Neutral, 2=Positive) are later mapped to their corresponding string labels so the distribution of sentiment can be effectively displayed in our final data visualizations.

Feature	**Description**	**Input Type**
text	The raw text of the social media post	string
label	The sentiment classification associated with the tex	int = [0, 1, 2]

2.4 Scraped Social Media Data (Bluesky)

For the demonstration, the project extracted social media posts from Bluesky using the AT Protocol API. The query targeted discussions surrounding the topic of "climate change" from January 2025 to February 2025. After extensive preprocessing to handle missing values and filter bot activity, the dataset was structured as a chronological time-series.

Field	Description
keyword	search term utilized for the API query
post_uri	URI for post
cid	Content identifier for post record
author_did	DID (Decentralized Identifier) of the author of the tweet
author_handle	Username of bluesky account
post_text	Raw string text content of the Bluesky post
created_at_raw	Raw chronological creation date of post
created_at_iso	ISO format timestamp of post creation timestamp
indexed_at	Timestamp for when it was processed into Bluesky system
reply_parent_uri	URI of direct parent post if post is a reply
reply_root_uri	URI of the root post if post is a reply
like_count	Number of likes on post
repost_count	Number of repost on post
reply_count	Number of replies on post
quote_count	Number of quotes connected to post
source_url	URL of post

2.5 Train, Validation, and Test Split

2.5.1 Bot-Detection dataset

For the bot-detection task, the data was pre-split into 81% training, 9% validation, and 10% test splits. The test set was not used during hyperparameter selection.

Split	# of rows	Purpose
Training	20,712	Used to fit model parameters and 5-fold CV for hyperparameter tuning
Validation	2,302	Was not used
Test	2,558	Held out until final evaluation to estimate generalization performance.

During the 5-fold Cross-Validation process, stratification was applied to ensure that each fold preserved a consistent proportion of human and bot examples. This is a critical step, as evaluation metrics can become heavily skewed and misleading if the validation folds contain an imbalanced class distribution.

2.5.2 Sentiment Analysis Dataset dataset

For the sentiment analysis dataset, the data split was quite imbalanced and came pre-split at 76% train / 3% val / 21% test.

Split	# of rows	Purpose
Training	45,615	Used to fit model parameters.
Validation	2000	Used through hugging face Trainer + Optuna to monitor validation loss / metrics after each epoch during hyperparameter optimization.
Test	12,284	Held out until final evaluation to estimate generalization performance.

3. Preprocessing and Feature Engineering

3.1 Text Cleaning

3.1.1 Bot Identification Dataset

No cleaning was applied to the bot identification dataset. Raw text was intentionally preserved because bot-generated content often contain specific structural anomalies such as irregular punctuation, unnatural spacing, or casing errors. These artifacts serve as critical predictive features for our model. Applying standard text cleaning (like lowercasing or punctuation removal) would destroy the nuances the filter needs.

3.1.2 Sentiment Analysis Dataset

The **tweet_eval** dataset was provided pre-processed by the original authors. Their standardization pipeline included anonymizing user mentions (replaced with @user), removing line breaks, and completely stripping out website URLs. We intentionally avoided applying any further manual cleaning to this dataset. Over-cleaning social media text—such as removing emojis, capitalization, or expressive punctuation (e.g., "!!!")—strips away vital contextual clues required for accurate sentiment classification.

3.1.3 Bluesky Dataset

Before modeling, the raw, unstructured data extracted from the Bluesky AT Protocol required structural formatting to match our pipeline's expected inputs:

- All metadata columns except post_text and created_at_raw were dropped.
- Rows containing missing values in either the text or timestamp fields were dropped.
- The post_text feature was explicitly cast to a string type.
- The created_at_raw timestamps were parsed and cast into standard DateTime objects within a new date column to enable chronological time-series grouping.

3.2 TF-IDF Representation (Bot Detection)

TF-IDF stands for term frequency-inverse document frequency. It converts text into numeric vectors where each feature represents a word, phrase, or character pattern. A term receives a higher value when it appears frequently in a document, but its importance is reduced if it appears in many documents across the full dataset. This helps the model focus on terms that are informative instead of terms that are simply common everywhere.

3.3 Word-Level TF-IDF (Bot Detection)

Word-level TF-IDF captures the vocabulary and short phrases used in a post. For bot detection, this is useful because bot-like accounts may repeatedly use certain topic words, promotional

language, slogans, or generic phrases. Word n-grams also allow the model to capture short phrases instead of only individual words.

3.4 Character-Level TF-IDF (Bot Detection)

Character-level TF-IDF captures small character lexical patterns instead of full words. This is useful for detecting writing-style patterns such as repeated punctuation, unusual spacing, repeated prefixes or suffixes, and generated-text artifacts. Character features can help identify bot-like text even when the exact words vary, because the style or formatting pattern may still be repeated.

3.5 RoBERTa Tokenization (Sentiment Analysis)

We utilized RoBERTa's Byte-Level BPE (Byte-Pair Encoding) Tokenizer, which is uniquely advantageous for noisy social media data. While standard BERT models use WordPiece tokenization that often discards emojis and informal characters as unknown ([UNK]) tokens, RoBERTa maps these complex symbols directly to their underlying 4-byte representations, preserving them as printable Unicode. Additionally, it maps special characters such as spaces and apostrophes to consistent Latin characters (for example, encoding leading spaces as the Ġ symbol). This precise mapping ensures that exact word boundaries, punctuation structures, and vital emotional nuances are retained without destructive pre-processing, significantly increasing the model's semantic correlation and contextual knowledge. The text is processed through the following steps:

1. **Byte Mapping & Merging:** Raw strings are converted into UTF-8 bytes and mapped to Unicode characters. These characters are iteratively merged into sub-word tokens based on learned frequency rules (e.g., contiguous 't' and 'h' characters are merged into a single 'th' token).
2. **ID Mapping:** Once fully merged, the token sequence is mapped to numerical IDs referencing RoBERTa's pre-trained vocabulary dictionary.
3. **Padding and Truncation:** Special start and end tokens are appended to the sequence. The sequence is then padded with null tokens (or truncated) to reach a uniform maximum sequence length (128 tokens). This standardization is strictly required for efficient matrix multiplication on GPU hardware.
4. **Attention Masking:** An attention mask vector is generated alongside the input IDs, assigning a 1 to actual text tokens and a 0 to padding tokens. This explicitly instructs the transformer mechanism to ignore artificial padding during its contextual calculations.

4. Models

4.1 Model Overview

Model	Input Representation	Task
XGBoost	Word + character TF-IDF	Bot detection
Logistic Regression	Word + character TF-IDF	Bot detection

Linear SVM	Word + character TF-IDF	Bot detection
RoBERTa	Tokenized text	Sentiment analysis

4.2 XGBoost

XGBoost (Extreme Gradient Boosting) is a highly optimized decision-tree ensemble model. It builds a sequence of trees where each new tree sequentially minimizes the residual errors made by the previous trees. For text classification, XGBoost receives sparse TF-IDF feature matrices and learns splits based on words, phrases, and character patterns. Unlike purely linear models, XGBoost can capture complex, non-linear interactions between features. This is particularly advantageous for bot detection, as automated text is rarely identified by a single isolated word, but rather, it is exposed through combinations of repeated vocabulary, rigid formatting, and specific stylistic patterns.

4.3 Logistic Regression

Logistic Regression serves as our linear classification baseline. It computes a linear combination by learning one weight for each TF-IDF feature, and then passes that aggregated score through a sigmoid function to output a bounded probability of belonging to the bot or human class. Logistic Regression is a strong, robust baseline for text classification because TF-IDF transformations often render text data highly linearly separable. Furthermore, it is computationally efficient and highly interpretable, as the learned feature weights directly indicate which specific words or character patterns push predictions toward a given class.

4.4 Linear SVM

Linear SVM finds a linear decision boundary that separates TF-IDF vectors into classes. It chooses the boundary with the widest margin between the closest points from each class. This works well for sparse, high-dimensional text data. In this project, Linear SVM is a reasonable model because the difference between human and bot text may be spread across many small word and character-pattern signals.

4.5 RoBERTa

RoBERTa is a transformer-based language model based on BERT. It uses masked language modeling during pretraining and removes BERT's next sentence prediction objective. RoBERTa is useful for sentiment analysis because it produces contextual embeddings. This means the representation of a word depends on surrounding words, not just the word by itself.

Self-attention allows each token to attend to other tokens in the input sequence. This helps the model understand modifiers and negation. For example, the phrase "not good" should not be treated the same as "good." A TF-IDF model may recognize both words, but RoBERTa is better suited to modeling how the words interact in context.

1. Input text is processed into token IDs and attention masks by the BPE tokenizer.
2. Token IDs and attention masks are passed into RoBERTa.

3. Multiple transformer encoder layers update each token's representation using multi-head self-attention.
4. A pooled output is passed to a dense classification head, which outputs raw logits for the sentiment classes (Negative, Neutral, Positive).
5. These logits are converted into probabilities, and the class with the highest probability is assigned as the definitive sentiment label

5. Training Objectives and Loss Functions

Model	Training Objective or Loss	Explanation
XGBoost	Binary logistic loss	Optimizes an additive ensemble of decision trees using gradient descent to iteratively minimize binary classification error.
Logistic Regression	Logistic loss with regularization	Learns linear feature weights to estimate the probability of a text being bot-generated, constrained by a penalty to prevent overfitting on sparse TF-IDF features.
Linear SVM	Hinge or squared hinge loss	Heavily penalizes misclassifications to mathematically optimize the maximum-margin hyperplane separating human and bot feature vectors.
RoBERTa sentiment model	Categorical cross-entropy loss	Minimizes the divergence between the model's predicted Softmax probability distribution (Negative, Neutral, Positive) and the true target labels.

6. Hyperparameter Search Space

6.1 Shared TF-IDF Hyperparameters

Hyperparameter	What It Controls	Values Tested
word_tfidf__ngram_range	Whether the word TF-IDF uses individual words or n-word phrases	(1,1), (1,2)
word_tfidf__max_features	Maximum number of word-based TF-IDF features	20k, 50k, 80k, 100k depending on model
word_tfidf__min_df	Removes words or phrases that appear in too few documents	2, 3, 5

word_tfidf__max_df	Removes words or phrases that appear in too many documents	0.90, 0.95, 1.0
word_tfidf__sublinear_tf	Uses log scaling for repeated word counts	True
char_tfidf__ngram_range	Controls the length of character patterns	(3,5), (4,6)
char_tfidf__max_features	Maximum number of character-based TF-IDF features	20k, 50k, 80k, 100k depending on model
transformer_weights	Controls the relative importance of word and character TF-IDF features	Multiple word/character weighting combinations

6.2 XGBoost Hyperparameters

Hyperparameter	What It Controls	Values Tested
n_estimators	The total number of sequential decision trees constructed in the boosting ensemble	300, 500, 800
max_depth	Restricts the maximum depth of individual trees, limiting overfitting on highly specific, isolated feature interactions	3, 4, 5, 6, 8
learning_rate	The step-size shrinkage used during updates to prevent overfitting and scales contribution of each subsequent tree	0.01, 0.03, 0.05, 0.1
subsample	The ratio of training instances randomly sampled to grow each tree, introducing stochasticity to improve generalization	0.7, 0.8, 0.85, 0.9
colsample_bytree	The subsample ratio of columns evaluated when constructing each individual tree	0.55, 0.6, 0.7, 0.8
gamma	The minimum loss reduction required to make a further partition on a leaf node and acts as a conservative pruning threshold	0, 0.1, 0.3, 0.5

min_child_weight	The minimum sum of instance weight needed in a child node, preventing the creation of leaves representing isolated anomalies	1, 2, 3, 5
reg_alpha	L1 regularization penalty on leaf weights, encouraging sparsity in feature selection	0, 0.01, 0.1, 0.3, 1
reg_lambda	L2 regularization penalty on leaf weights, encouraging smoother, more conservative weight distributions	1, 5, 10, 20
scale_pos_weight	Weight multiplier applied to the positive class, crucial for navigating imbalanced class distributions	1, 2, 2.5, 5
max_bin	The maximum number of discrete bins used to bucket continuous features, optimizing computational speed and memory allocation	64, 128, 256

6.3 Linear SVM Hyperparameters

Hyperparameter	What It Controls	Values Tested
penalty	The regularization norm (L1 or L2) applied to the feature weights to enforce sparsity or smoothness	l1, l2
loss	The specific mathematical formulation of the loss function (e.g., standard hinge vs. squared hinge loss)	squared_hinge, hinge
dual	Toggles whether the algorithm solves the dual or primal optimization problem (primal is strictly preferred when n_samples > n_features)	True, False
C	The inverse of regularization strength; smaller values enforce strict regularization by creating a wider margin, at the cost of more training errors	0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1, 3, 10

class_weight	Dynamically adjusts the misclassification penalty per class, mitigating biases caused by dataset imbalance	None, balanced
tol	The stopping tolerance; the algorithm terminates when loss updates fall below this threshold	0.0001, 0.001
max_iter	The hard limit on training iterations, preventing infinite loops if the data is not perfectly separable	5000, 10000, 20000

6.4 Logistic Regression Hyperparameters

Hyperparameter	What It Controls	Values Tested
penalty	The regularization technique applied: L1 (Lasso), L2 (Ridge), or Elastic Net	l2, l1, elasticnet
C	The inverse regularization coefficient; smaller values aggressively shrink feature weights to prevent overfitting	loguniform(1e-3, 100)
l1_ratio	The mixing parameter for Elastic Net; dictates the exact ratio between L1 sparsity and L2 smoothness	0.1, 0.25, 0.5, 0.75, 0.9
class_weight	Adjusts the loss function to heavily penalize errors on minority classes	None, balanced
tol	The convergence threshold for the optimization algorithm	0.001, 0.0001
max_iter	The maximum number of epochs the solver is allowed to run before forced termination	1000, 2000, 3000

6.5 RoBERTa Hyperparameters (Optuna + Trainer)

Hyperparameter	What It Controls	Ranges Tested
learning_rate	The peak step-size used by the AdamW optimizer to update the neural network's weights	[0.00001, 0.00005]

per_device_train_batch_size	The number of text sequences processed simultaneously before a weight update	[16, 32, 64]
weight_decay	Applies an L2 penalty to the network's weights during optimization, actively suppressing memorization and overfitting	[0.0, 0.1]
warmup_ratio	The fraction of total training steps used to linearly increase the learning rate from zero to its peak, preventing catastrophic divergence in early epochs	[0.0, 0.2]
num_train_epochs	The total number of complete passes through the training dataset before termination	[2, 5]

7. Evaluation Metrics

7.1 Bot Detection

All final bot-detection performance metrics in this report are evaluated exclusively on the held-out test set to ensure an unbiased estimate of generalization. The test set contains 2,558 total examples, perfectly balanced between 1,278 authentic examples (Class 0: Human) and 1,280 adversarial examples (Class 1: Bot). The F1-score was designated as the primary metric for final model selection.

Metric	Meaning
F1-score	Harmonic mean of precision and recall

7.1.1 Why F1-Score Was Used for Model Selection

The F1-score was selected because the integrity of the downstream sentiment index relies heavily on minimizing both false positives and false negatives. Prioritizing precision alone ignores missed bots (false negatives), which would artificially inflate or deflate the sentiment index. Conversely, optimizing purely for recall disregards false positives, leading to the systemic exclusion of authentic human discourse. The F1-score forces the model to balance these competing priorities, making it a far more informative performance indicator than raw accuracy when the operational cost of both error types is high.

7.2 Sentiment Analysis

Metric	Meaning

Macro Average Recall	Calculates the recall for each distinct class independently and averages them without weighting by class frequency

7.2.2 Why Macro Average Was Used For Model Selection

Following the official benchmark guidelines established by the authors of the tweet_eval dataset, Macro Average Recall was utilized as the primary evaluation metric. This choice is mathematically necessary due to the significant class imbalances present in the sentiment dataset. Standard Accuracy or a Micro-Averaged metric would be disproportionately influenced by the majority class (e.g., if "Neutral" dominates the dataset, the model could achieve high accuracy simply by guessing "Neutral" every time). By calculating the unweighted average of the recall across all three classes, Macro Average Recall ensures that the model is held equally accountable for its performance on minority classes (such as "Negative" sentiment) as it is for the majority class.

8. Hyperparameter Optimization Method

8.1 Bot Identification Model

Hyperparameter tuning was performed using RandomizedSearchCV. Each model tested 50 randomly sampled hyperparameter combinations. Each sampled configuration was evaluated using 5-fold stratified cross-validation. The scoring metric used to select the best model was F1-score with the positive label set to 1, denoting bot-generated content.

Model	**n_iter**	**Scoring**	CV	**n_jobs**	**pre_dispatch**
XGBoost	50	F1 (bot=1)	5-fold stratified CV	5	5
Linear SVM	50	F1 (bot=1)	5-fold stratified CV	20	20
Logistic Regression	50	F1 (bot=1)	5-fold stratified CV	30	10

8.2 Optimal Configurations (Bot Detection)

8.2.1 XGBoost

XGBoost Parameter	**Best Value**
n_estimators	500
max_depth	6
learning_rate	0.1
subsample	0.9
colsample_bytree	0.7
gamma	0.1
min_child_weight	3

reg_alpha	0.1
reg_lambda	10
scale_pos_weight	2.5
max_bin	64
word_tfidf__ngram_range	(1,1)
char_tfidf__ngram_range	(3,5)
transformer_weights	word_tfidf=1.0, char_tfidf=0.5

8.2.2 Linear SVM

Linear SVM Parameter	Best Value
penalty	l1
loss	squared_hinge
dual	False
C	0.3
class_weight	None
tol	0.001
max_iter	10000
word_tfidf__max_features	50000
char_tfidf__max_features	50000
transformer_weights	word_tfidf=1.0, char_tfidf=1.25

8.2.3 Logistic Regression

Logistic Regression Parameter	Best Value
penalty	l1
C	2.3046
class_weight	balanced
tol	0.001
max_iter	1000
word_tfidf__ngram_range	(1,2)
word_tfidf__max_features	80000
char_tfidf__ngram_range	(3,5)
char_tfidf__max_features	80000
transformer_weights	word_tfidf=1.0, char_tfidf=1.25

8.3 Sentiment Analysis Model

Unlike the traditional machine learning models that utilized randomized search, the RoBERTa transformer requires navigating a highly sensitive, continuous hyperparameter space. To efficiently optimize this architecture without exhausting computational resources, Bayesian optimization was deployed via the Optuna framework. The search monitored validation performance across multiple trials, ultimately isolating the following configuration as the optimal setup for the sequence classification head:

RoBERTa Parameter	Best Value
learning_rate	3.55648335518932e-05
per_device_train_batch_size	64

weight_decay	0.013193237590233865
warmup_ratio	0.16999496770092737
num_train_epochs	2

9. Results

9.1 Bot Identification

9.1.1 Main Model Comparison

The table below compares final model performance on the held-out test set.

Model	Test Accuracy	Macro F1	Weighted F1	Bot-Class F1
XGBoost	0.8393	0.8368	0.8368	0.8570
Logistic Regression	0.8370	0.8368	0.8368	0.8421
Linear SVM	0.8393	0.8385	0.8385	0.8501

XGBoost and Linear SVM tied for the highest overall test accuracy at 0.8393. Linear SVM marginally achieved the highest test Macro F1-score (0.8385), suggesting it maintained the most mathematically balanced performance across both classes. However, XGBoost achieved the highest test Bot-Class F1-score (0.8570), indicating it is the most aggressive and capable model when the primary objective is to maximize the extraction of adversarial content.

9.1.2 XGBoost Test Performance

Class	Precision	Recall	F1-Score	Support
Human / 0	0.9502	0.7160	0.8166	1278
Bot / 1	0.7724	0.9625	0.8570	1280

XGBoost exhibited exceptional sensitivity to adversarial text, achieving a near-perfect Bot Recall of 0.9625 (successfully identifying 96.25% of all actual bot posts in the test set). The trade-off for this aggressive filtering was a lower Human Recall of 0.7160 on the test set, indicating a higher rate of false positives (authentic human posts incorrectly flagged as bots). While this results in data loss, it is a highly acceptable trade-off for a pipeline designed to prioritize the purity of downstream sentiment analysis.

9.1.3 Logistic Regression Test Performance

Class	Precision	Recall	F1-Score	Support
Human / 0	0.8596	0.8052	0.8315	1278
Bot / 1	0.8170	0.8688	0.8421	1280

Logistic Regression proved to be the most symmetrical model. While it did not match XGBoost's aggressive bot recall, it produced significantly less extreme variances between the classes. Furthermore, because Logistic Regression directly assigns interpretable weights to individual

TF-IDF features, it remains a highly valuable baseline model for understanding exactly which lexical patterns drive bot classifications.

9.1.4 Linear SVM Test Performance

Class	Precision	Recall	F1-Score	Support
Human / 0	0.8952	0.7684	0.8269	1278
Bot / 1	0.7974	0.9102	0.8501	1280

Linear SVM performed comparably to XGBoost in raw accuracy but achieved the highest overall Macro F1-score. This indicates that the maximum-margin hyperplane offered the strongest overall statistical balance between preserving human text and catching bots. The high performance of the SVM strongly validates the hypothesis that TF-IDF representations are highly suitable for this task, as the bot/human distinction is successfully separated by evaluating thousands of subtle lexical and stylistic signals simultaneously.

9.1.5 Model Selection

For the final production pipeline, **XGBoost** was selected as the primary bot identification filter. The overarching architecture of this project prioritizes data purity; it is critical to aggressively remove bot-like and adversarial content *before* performing sentiment translation. This is especially vital for highly targeted, socially significant topics (such as climate change), where organized bot networks deliberately attempt to artificially distort measured public sentiment. Because the Bluesky API allows for high-volume data extraction, the system can easily withstand the loss of some authentic human rows (due to XGBoost's lower human recall) in exchange for near-absolute certainty that the surviving dataset is devoid of bot interference.

9.2 Sentiment Analysis

9.2.1 Overall Model Metrics

The fine-tuned RoBERTa-base model was evaluated on the strictly held-out test set consisting of 12,284 examples. As established in the project objectives, the primary metric for evaluation is **Macro Average Recall** to account for the class imbalance within the social media corpus.

Metric	Test Set Value
Test Accuracy	0.7129
Macro Average Recall	0.7330
Macro Average F1-Score	0.7135
Macro Average Precision	0.7024

9.2.2 Class-Level Metrics (Test set)

Sentiment Class	Precision	Recall	F1-Score
Negative (0)	0.7116	0.7777	0.7432
Neutral (1)	0.7553	0.6436	0.6950
Positive (2)	0.6404	0.7777	0.7024

On the test set, the model achieved a Macro Average Recall of 0.7330 and a Macro F1-score of 0.7135. These results are significant as they align closely with state-of-the-art benchmarks for the tweet_eval dataset, which typically range between 0.70 and 0.72 for base transformer architectures.

The model demonstrated its highest test F1-score on the Negative class (0.7432), which is crucial for our objective of identifying sentiment dips and public dissatisfaction on Bluesky. While the test recall for the Neutral class (0.6436) was lower than the other categories, the high test precision (0.7553) for that same class indicates that when the model predicts a neutral sentiment, it is highly likely to be correct. This balanced performance across all three labels on the test set ensures that the resulting sentiment drift visualizations are a reliable reflection of actual public discourse rather than statistical noise.

10. Sentiment Drift Visualization

10.1 Sentiment Scoring

For the sentiment drift portion of the project, each collected social media post receives a sentiment label or numeric sentiment score. The model outputs logits for sentiment classes. These logits are converted into probabilities, and the class with the highest probability becomes the predicted sentiment. The sentiment label is then mapped into a numeric score so the values can be plotted over time.

Sentiment	Numeric Mapping
Negative	0
Neutral	1
Positive	2

10.2 Time-Series Aggregation

After each post is assigned a sentiment score, posts are grouped by timestamp to create a time-series view of public sentiment. For each time point, the project plots the number of posts,

the average sentiment score, and a smoothed average sentiment curve to show both short-term fluctuations and longer-term sentiment trends.

10.3 Output

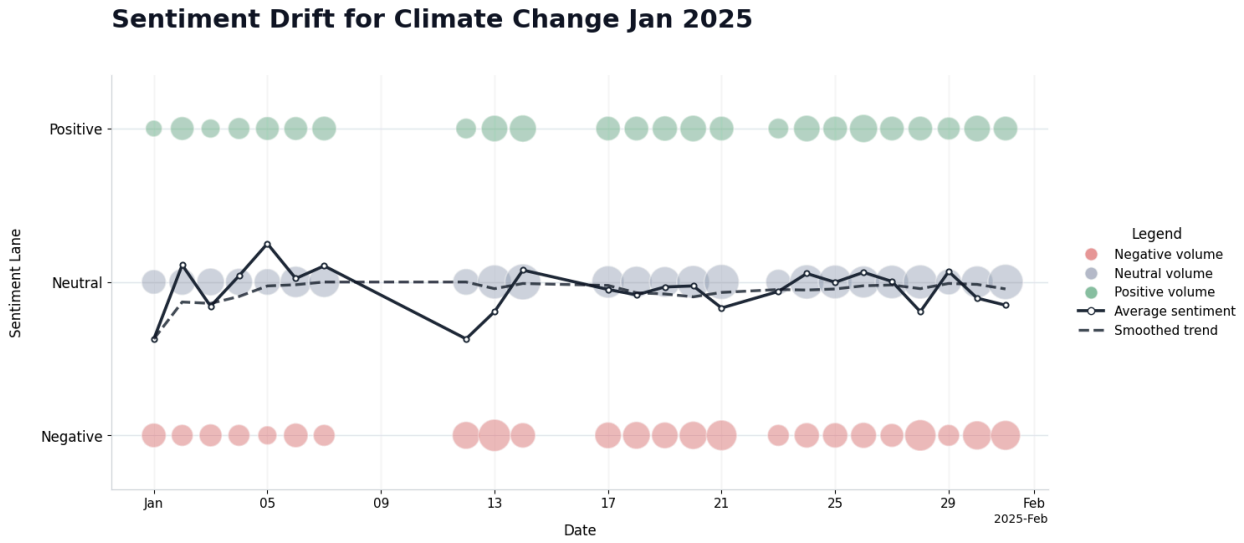

Figure 1. Sentiment Drift Visualization.

Bubble size represents the number of posts classified as negative, neutral, or positive at each time point. The solid line shows average sentiment, while the dashed line shows the smoothed trend.

10.4 Interpreting Sentiment Spikes and Dips

Major rises or drops in sentiment should be interpreted carefully. A sentiment spike may coincide with a real-world event, but the graph alone does not prove causation. The report should use cautious language such as "may be associated with" or "coincides with." Any event explanation should be supported with a citation to a relevant news article, platform status page, or other trustworthy source.

Date Range	Sentiment Change	Possible Explanation	Evidence Needed
Jan 7 - Jan 13	Drop	Coverage of LA wildfires may be associated with sentiment drop	Winds, warmth and relentless drought fueled Los Angeles fires, scientists say, Los Angeles wildfires devour thousands of homes, death toll rises to 10

11. Final Pipeline and Implementation

11.1 Sentiment Drift Pipeline

The final sentiment drift pipeline takes a user-selected keyword or topic and produces a time-series visualization of sentiment. The steps are as follows:

1. User enters a keyword or topic
2. The scraper collects posts and timestamps from Bluesky
3. The posts are cleaned and converted into a consistent format
4. Botted post are identified and discarded by the bot identification model
5. The sentiment model assigns a sentiment label or score to each post
6. Sentiment scores are grouped by time
7. The system produces a sentiment drift graph and saves it as an image
8. Major rises or drops are interpreted using external event evidence

11.3 Repository Organization

Path	Purpose
data/bot_identification/	Data used to train and test the bot identification models
data/climate_change/	Raw climate change data retrieved from Bluesky for January 2025
notebooks/	Notebooks for training/testing models and pipeline
scripts/	Folder for Bluesky webscraping script
model_results/	Evaluation metrics and best hyperparameter results for models
sentiment_drift_graphs/	Output directory for sentiment graphs

12. Adopted Work vs. Original Work

Component	Adopted From Existing Work	Original Project Contribution
TweepFake dataset	Open-source deepfake text dataset containing balanced human and bot-generated tweets.	Utilized as the foundational dataset to train, validate, and evaluate the custom anomaly detection models.
TweetEval / RoBERTa sentiment resources	Pre-trained RoBERTa transformer weights and the standardized CardiffNLP social media sentiment dataset.	Fine-tuned the classification head via Bayesian optimization (Optuna) and integrated the inference engine into a continuous time-series pipeline.

Scikit-learn and XGBoost	Standardized open-source machine learning frameworks and mathematical solvers.	Engineered the custom TF-IDF feature extraction pipelines, executed rigorous stochastic hyperparameter optimization, and conducted comparative model evaluation.
Social media scraper/API	The external REST API endpoints and underlying data infrastructure provided by Bluesky.	Engineered the data ingestion architecture to dynamically extract, preprocess, and structure topic-specific temporal data (e.g., climate change) into a machine-readable format.
Sentiment drift visualization	Standard data visualization plotting libraries (e.g., Matplotlib / Seaborn / Pandas).	Designed the mathematical aggregation logic (rolling averages, numeric sentiment mapping) and created the final chronological visualization framework to correlate sentiment drift with real-world events.

13. Discussion

13.1 Why TF-IDF Was Reasonable for Bot Detection

TF-IDF was a reasonable representation for bot detection because bot-like behavior often appears in repeated vocabulary, repeated phrase patterns, formatting habits, and stylistic signals. Word TF-IDF captures what terms and phrases appear in the text, while character TF-IDF captures how the text is written. Together, they give the model information about both content and style.

The strong performance of Logistic Regression and Linear SVM supports this choice. These models work well with sparse, high-dimensional data, which is exactly the format produced by TF-IDF. XGBoost also performed well, especially for bot recall, because it could model combinations of TF-IDF features.

13.2 Why RoBERTa Was Used for Sentiment Analysis

Unlike bot detection, sentiment translation is highly dependent on sequential context and subtle semantic nuance. Traditional bag-of-words or TF-IDF representations frequently fail to capture negation (e.g., "not good"), sarcasm, and complex modifiers because they evaluate terms in isolation. The integration of the RoBERTa transformer effectively circumvents this limitation. Through multi-head self-attention and dynamic contextual embeddings, RoBERTa fundamentally alters the numerical representation of a token based on its surrounding

bidirectional context. This ability to map how words interact dynamically makes it vastly superior for decoding complex human emotional intent.

13.3 Why Not Use RoBERTa for Everything?

Given RoBERTa's advanced NLP capabilities, a common architectural question is why it was not deployed for the initial bot detection phase as well. This design decision is rooted in computational efficiency, diminishing returns, and model interpretability.

Transformer architectures introduce massive computational overhead (requiring heavy GPU parallelization). The project results demonstrate that deep, semantic contextual understanding is fundamentally unnecessary for identifying automated text as bots are reliably exposed through superficial lexical and stylistic footprints. Deploying lightweight, highly interpretable TF-IDF pipelines for the initial filtering phase minimizes compute costs, maximizes inference speed, and allows feature weights to be audited. This effectively reserves the heavy, computationally expensive RoBERTa engine exclusively for the highly nuanced task of sentiment translation.

13.4 Generalization

13.4.1 Covariance (Feature) Shift

Our models were trained on Twitter data (X), but are performing inference on Bluesky data. While both platforms utilize short-form text, Bluesky's "culture" and slang may differ from the 2017–2022 era of the training sets. However, because RoBERTa uses byte-level BPE, it should be robust enough to handle these slight shifts in vocabulary.

13.4.2 Label Shift

Label shift may occur if the proportion of bots on Bluesky is significantly lower than the 50/50 split in our training data. To mitigate this, we utilized scale_pos_weight in XGBoost and monitored Precision-Recall curves to ensure the model remains effective even when bots are a minority.

13.4.3 Concept Shift

Concept shift represents the risk that the definition of a "bot" or "positive sentiment" changes over time. For example, as LLMs are more common, the "bot" patterns in the TweepFake dataset (built on GPT-2) may become outdated. Continuous monitoring and periodic retraining on newer samples will be required to maintain long-term accuracy.

References

[1] TweepFake deepfake text detection dataset. GitHub repository: https://github.com/tizfa/tweepfake_deepfake_text_detection

[2] TweetEval / CardiffNLP benchmark resources. GitHub repository: https://github.com/cardiffnlp/tweeteval

[3] Scikit-learn documentation: TfidfVectorizer, LogisticRegression, LinearSVC, StratifiedKFold, and RandomizedSearchCV.

[4] XGBoost documentation: XGBClassifier parameters and training objectives.

[5] Hugging Face Transformers documentation: RoBERTa models, tokenizers, logits, and sequence classification.

[6] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). *RoBERTa: A robustly optimized BERT pretraining approach*. arXiv. https://arxiv.org/abs/2008.0003

[7] Nguyen, D. Q., Vu, T., & Nguyen, A. T. (2020). *BERTweet: A pre-trained language model for English tweets*. arXiv. https://arxiv.org/pdf/2010.12421

[8] Rosenthal, S., Farra, N., & Nakov, P. (2017). *SemEval-2017 Task 4: Sentiment analysis in Twitter*. In *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)* (pp. 502–518). https://aclanthology.org/S17-2088/